

Java Garbage Collection & Memory Issues – Quick Revision

Does Java Process Memory Go Beyond -Xmx?

- Yes, total process memory includes heap (-Xmx) + metaspace + thread stacks + direct + native allocations.
- Monitoring tools may show usage greater than -Xmx.

Out Of Memory Error Types

- Java heap space
- GC overhead limit exceeded
- Requested array size exceeds VM limit
- Metaspace / PermGen space
- Direct buffer memory
- Unable to create new native thread

Common Causes of OOM

- Memory leaks (unreleased references)
- Very large data structures
- Uncontrolled class loading
- Too many threads
- High direct memory usage

GC Behavior Patterns

- Healthy app: Heap usage oscillates after GC cycles
- Acute leak: Rapid heap growth, frequent Full GCs
- Gradual leak: Slow heap growth, increasing GC frequency

Key Diagnostics

- GC Logs → Analyze with GCeasy
- Heap Dump → Eclipse MAT / HeapHero
- Thread Dump → jstack / kill -3

Special OOM Cases

- GC overhead limit exceeded – >98% time in GC, <2% heap freed
- Requested array size exceeds VM limit – allocation beyond Integer.MAX_VALUE
- Metaspace – Java 8+ class metadata storage, limited by -XX:MaxMetaspaceSize
- PermGen Space – pre-Java 8 class metadata
- Unable to create new native threads – OS/thread stack limits
- Direct buffer memory – off-heap via ByteBuffer.allocateDirect()
- Kill Process or Sacrifice Child – Linux OOM killer terminating JVM

Real Case Studies

- Major US financial institution – slowdown due to threads blocked on I/O
- AWS EBS HTTP 502 errors – native memory exhaustion from direct buffers